

TWP Assignment 03

Article Summarization

System Correctness Practices at Amazon Web Services

MT26MCS024 - Gaurav Acharya

MT26MCS032 - Sandeep

September 2026

1 Summarization

The historical approach to maintaining system correctness used by Amazon Web Services (AWS) was to use a systematic and high-performance optimization language called TLA $\pm$, which allowed developers to identify and eliminate bugs during very early development, but it was very hard to understand and implement development of another language formally named P Language was done. The P Language sought a balance between mathematical thinking capabilities while being easier to understand by using state-machine-based algorithms, allowing developers to model and analyze the distributed systems.

Some of the crucial works validated using P Language included are; Migration of Simple Storage Service(S3) to a more strong read-after-write consistency by using several non-trivial changes to the S3 index protocol stack. In 2023 AWS built a new tool PObserve which validated the correctness of distributed systems during both testing and production.

AWS also used multiple lightweight formal methods closer to its engineering team: **Property-based testing** - key idea is that team combines property-based testing with developer-provided correctness specifications, coverage-guided fuzzing, failure injection, and minimization; **Deterministic simulation** - distributed system executes on a single-threaded simulator with control over all sources of randomness; **Continuous fuzzing or random test-input generation** -covers guided scalable test-input generation,effective for testing systems correctness at integration time.

AWS launched Fault Injection Service allowing customers to validate that the resiliency mechanisms, improving availability and do not introduce correctness problems. While fault injection is not considered a formal method, it can be combined with formal specifications. There has been an emerging interest in a particular class of systems failure: those where some triggering event causes a distributed system to enter a state where it does not recover without intervention. This class of failures, dubbed metastable failures,⁵ is one of the most important contributors to unavailability in cloud systems.

Formal methods are a crucial part of safely improving the performance of cloud systems. Productivity and cost benefits, the low-level code that normally ignored. Using the HOL Light interactive theorem prover, the team was able to prove the correctness of these optimizations. This reduces infrastructure costs and aid sustainability while at the same time improving customer-visible performance.

The primary barriers for formal methods tools are steep learning curve, required specialized domain expertise and lack of user-friendly interfaces. The challenge is educating distributed-systems developers about these testing methods, graduates have not been taught the basics of formal reasoning approaches. Metastability facing similar awareness challenges. limited technology making system stability modeling expensive and complex.